# LAB SHEET - 8

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import HTML
from base64 import b64encode

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
video_path = "/content/optical_flow.mp4"

print("Video path:", video_path)
```

```
Video path: /content/optical_flow.mp4
```

```
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open the video.")
else:
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    duration = frame_count / fps if fps > 0 else 0

    print("Video opened successfully!")
    print("Width:", width)
    print("Height:", height)
    print("FPS:", fps)
    print("Total Frames:", frame_count)
    print("Duration: {:.2f} seconds".format(duration))

cap.release()
```

```
Video opened successfully!
Width: 640
Height: 360
FPS: 30.0
Total Frames: 90
Duration: 3.00 seconds
```

```
video_file = open(video_path, "rb").read()

video_url = "data:video/mp4; base64," + b64encode(video_file).decode()

HTML(f"""
<video width="700" controls>
    <source src="{video_url}" type="video/mp4">
</video>
""")
```

```
cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()

if not ret:
    print("Error: Could not read the first frame.")
else:
    print("First frame read successfully!")

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

print("Feature points detected:", len(p0) if p0 is not None else 0)
```

```
First frame read successfully!
Feature points detected: 31
```

```
mask = np.zeros_like(old_frame)

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*'mp4v')

output_lk = "/content/lucas_kanade_tracking.mp4"

out = cv2.VideoWriter(
    output_lk,
    fourcc,
    fps,
    (width, height)
)

total_displacement = 0
motion_count = 0
speed_values = []

while True:

    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    if p0 is not None and len(p0) > 0:

        p1, status, error = cv2.calcOpticalFlowPyrLK(
```

```
                old_gray,
                frame_gray,
                p0,
                None,
                **lk_params
            )

            if p1 is not None:

                good_new = p1[status == 1]
                good_old = p0[status == 1]

                frame_displacement = 0

                for new, old in zip(good_new, good_old):

                    x_new, y_new = new.ravel()
                    x_old, y_old = old.ravel()

                    displacement = np.sqrt(
                        (x_new - x_old)**2 +
                        (y_new - y_old)**2
                    )

                    frame_displacement += displacement

                    mask = cv2.line(
                        mask,
                        (int(x_new), int(y_new)),
                        (int(x_old), int(y_old)),
                        (0, 255, 0),
                        2
                    )

                    frame = cv2.circle(
                        frame,
                        (int(x_new), int(y_new)),
                        5,
                        (0, 0, 255),
                        -1
                    )

                    cv2.arrowedLine(
                        frame,
                        (int(x_old), int(y_old)),
                        (int(x_new), int(y_new)),
                        (255, 0, 0),
                        2,
                        tipLength=0.3
                    )

                if len(good_new) > 0:

                    avg_displacement = (
                        frame_displacement /
                        len(good_new)
                    )

                    total_displacement += avg_displacement
                    motion_count += 1

                    speed_values.append(
                        avg_displacement * fps
                    )

                p0 = good_new.reshape(-1, 1, 2)

                if len(p0) < 10:

                    p0 = cv2.goodFeaturesToTrack(
                        frame_gray,
                        mask=None,
                        **feature_params
                    )

        result = cv2.add(frame, mask)
```

```
        cv2.putText(
            result,
            "Lucas-Kanade Optical Flow",
            (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.9,
            (255, 255, 255),
            2
        )

        out.write(result)

        old_gray = frame_gray.copy()

    cap.release()
    out.release()

    print("Lucas-Kanade tracking completed!")
    print("Output:", output_lk)
```

```
Lucas-Kanade tracking completed!
Output: /content/lucas_kanade_tracking.mp4
```

```
video_file = open(output_lk, "rb").read()

video_url = "data:video/mp4; base64," + b64encode(video_file).decode()

HTML(f"""
<video width="700" controls>
    <source src="{video_url}" type="video/mp4">
</video>
""")
```

```
cap = cv2.VideoCapture(video_path)

ret, first_frame = cap.read()

if not ret:
    print("Error: Could not read video.")
else:
    print("Video opened successfully!")

prev_gray = cv2.cvtColor(
    first_frame,
    cv2.COLOR_BGR2GRAY
)

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*'mp4v')

output_fb = "/content/farneback_tracking.mp4"

out = cv2.VideoWriter(
    output_fb,
    fourcc,
    fps,
    (width, height)
)

total_magnitude = 0
frame_count_fb = 0
speed_values_fb = []
```

```
Video opened successfully!
```

```
while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    avg_magnitude = np.mean(magnitude)

    total_magnitude += avg_magnitude
    frame_count_fb += 1

    speed_values_fb.append(
        avg_magnitude * fps
    )

    hsv = np.zeros_like(frame)

    hsv[..., 1] = 255

    hsv[..., 0] = (
        angle * 180 / np.pi / 2
    )

    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    flow_visual = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.putText(
        flow_visual,
        "Farneback Dense Optical Flow",
        (20, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.9,
        (255, 255, 255),
        2
    )

    out.write(flow_visual)

    prev_gray = gray.copy()

cap.release()
out.release()
```

```
print("Farneback optical flow completed!")
print("Output:", output_fb)
```

```
Farneback optical flow completed!
Output: /content/farneback_tracking.mp4
```

```
video_file = open(output_fb, "rb").read()

video_url = "data:video/mp4; base64," + b64encode(video_file).decode()

HTML(f"""
<video width="700" controls>
    <source src="{video_url}" type="video/mp4">
</video>
""")
```

```
if motion_count > 0:

    average_displacement = (
        total_displacement /
        motion_count
    )

    average_speed_lk = np.mean(
        speed_values
    )

    print("Lucas-Kanade Motion Analysis")
    print("--------------------------------")
    print(
        "Average displacement per frame: {:.2f} pixels"
        .format(average_displacement)
    )

    print(
        "Average motion speed: {:.2f} pixels/second"
        .format(average_speed_lk)
    )

else:
    print("No motion detected.")
```

```
Lucas-Kanade Motion Analysis
--------------------------------
Average displacement per frame: 0.00 pixels
Average motion speed: 0.00 pixels/second
```

```
if frame_count_fb > 0:

    average_magnitude_fb = (
        total_magnitude /
        frame_count_fb
    )

    average_speed_fb = np.mean(
        speed_values_fb
    )

    print("Farneback Motion Analysis")
    print("--------------------------------")

    print(
        "Average motion magnitude: {:.2f} pixels"
        .format(average_magnitude_fb)
    )
```

```
        print(
            "Average motion speed: {:.2f} pixels/second"
            .format(average_speed_fb)
        )

    else:
        print("No motion detected.")
```

```
Farneback Motion Analysis
--------------------------------
Average motion magnitude: 0.10 pixels
Average motion speed: 2.93 pixels/second
```

```
plt.figure(figsize=(10, 5))

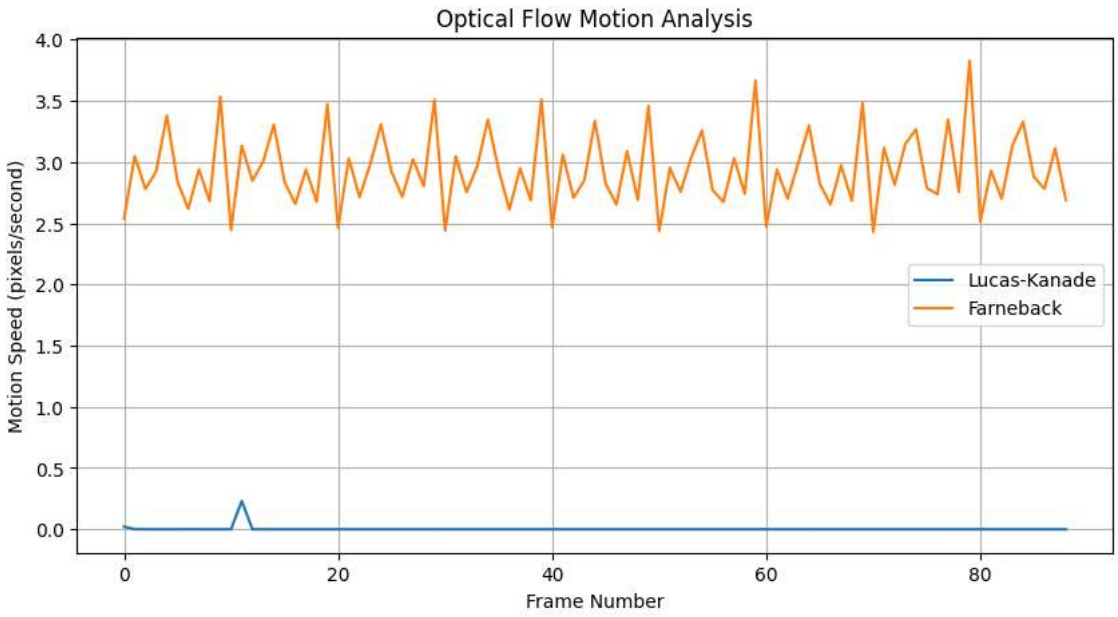
plt.plot(
    speed_values,
    label="Lucas-Kanade"
)

plt.plot(
    speed_values_fb,
    label="Farneback"
)

plt.xlabel("Frame Number")
plt.ylabel("Motion Speed (pixels/second)")
plt.title("Optical Flow Motion Analysis")

plt.legend()
plt.grid(True)

plt.show()
```




**Summary**

Method 1: Shi-Tomasi Corner Detection with Lucas-Kanade Sparse Optical Flow
Method 2: Farneback Dense Optical Flow
Input: Video sequence
Output: Motion vectors and tracked feature points
Motion Analysis: Object displacement, direction, and average motion/speed were analyzed.

**Result**

Optical flow successfully detected and tracked the motion of objects across consecutive video frames. Lucas-Kanade provided sparse tracking of selected feature points, while Farneback estimated dense motion across the image. The experiment demonstrated the application of optical flow for real-time object tracking and motion analysis.